

Case Study: Employee Task Tracker API (FastAPI + Database)

1. Overview

This case study is aiming to build a backend service using FastAPI with database connectivity. The task is to design and implement a small backend system demonstrating REST API design, ORM usage, and CRUD operations.

2. Project Title

Employee Task Tracker API

3. Problem Statement

Create an internal tool where employees can track the tasks assigned to them. The API should support employee creation, task assignment, listing tasks, updating task status, and database connectivity using ORM.

4. Requirements

4.1 Functional Requirements

- Allow adding new employees.
- Each employee can have multiple tasks.
- Task must contain title, status, and employee reference.
- Support adding new tasks.
- Support listing all tasks.
- Support listing tasks for a specific employee.
- Support updating task status.
- Input validation must be applied.
- Proper error handling is required.

4.2 Non-Functional Requirements

- Use FastAPI framework.
- Use SQLite or similar lightweight database.
- Use an ORM such as SQLAlchemy.
- Maintain clean project structure.
- Expose API documentation at /docs.

5. Entities to Be Created

5.1 Employee Entity

Field Name	Description
id	Unique identifier (auto-generated)
name	Name of the employee
tasks	List of tasks assigned to the employee

5.2 Task Entity

Field Name	Description
id	Unique task ID
title	Task name
status	Current status (default: Pending)
employee_id	ID of the employee the task belongs to

6. API Endpoints to Be Developed

Employee APIs:

- POST /employees/ - Create a new employee
- GET /employees/{employee_id}/tasks - Get all tasks of a specific employee

Task APIs:

- POST /tasks/ - Create a new task
- GET /tasks/ - Get all tasks in the system
- PUT /tasks/{task_id} - Update the status of a task

7. Constraints

- Employee name must not be empty.
- Task title must not be empty.
- Task status must be one of: Pending, In-Progress, Completed.
- Each task must be linked to a valid employee.

8. Deliverables

- Working FastAPI server

- Implemented CRUD operations
- Database tables using ORM
- Functional API documentation at /docs
- Optional: Postman Collection

9. Optional Extensions

- Add authentication (JWT)
- Add timestamps for tasks
- Add task priority
- Add pagination
- Add filtering (e.g., by status)